

PWDFT-SW: Extending the Limit of Plane-Wave DFT Calculations to 16K Atoms on the New Sunway Supercomputer

Qingcai Jiang¹, Zhenwei Cao, Junshi Chen¹, Xinming Qin, Wei Hu¹, Hong An¹, *Member, IEEE*, and Jinlong Yang¹

Abstract—First-principles density functional theory (DFT) with plane wave (PW) basis set is the most widely used method in quantum mechanical material simulations due to its advantages in accuracy and universality. However, a perceived drawback of PW-based DFT calculations is their substantial computational cost and memory usage, which currently limits their ability to simulate large-scale complex systems containing thousands of atoms. This situation is exacerbated in the new Sunway supercomputer, where each process is limited to a mere 16 GB of memory. Herein, we present a novel parallel implementation of plane wave density functional theory on the new Sunway supercomputer (PWDFT-SW). PWDFT-SW fully extracts the benefits of Sunway supercomputer by extensively refactoring and calibrating our algorithms to align with the system characteristics of the Sunway system. Through extensive numerical experiments, we demonstrate that our methods can substantially decrease both computational costs and memory usage. Our optimizations translate to a speedup of 64.8x for a physical system containing 4,096 silicon atoms, enabling us to push the limit of PW-based DFT calculations to large-scale systems containing 16,384 carbon atoms.

Index Terms—Quantum mechanics, plane-wave density functional theory, new Sunway supercomputer, parallel implementation.

Received 24 May 2024; revised 20 March 2025; accepted 29 March 2025. Date of publication 7 April 2025; date of current version 9 June 2025. This work was supported in part by the Strategic Priority Research Program of Chinese Academy of Sciences under Grant XDB0500102, Grant XDB0450101, and Grant XDB1170000, in part by the Laoshan Laboratory under Grant LSKJ202300305, in part by the Innovation Program for Quantum Science and Technology under Grant 2021ZD0303306, in part by the National Natural Science Foundation of China under Grant 22173093, Grant 22373096, and Grant 21688102, in part by the Anhui Provincial Key Research and Development Program under Grant 2022a05020052, in part by the National Key Research and Development Program of China under Grant 2021YFB0300600, in part by the CAS Project for Young Scientists in Basic Research under Grant YSBR-005, and in part by the Fundamental Research Funds for the Central Universities under Grant WK2320000061. Recommended for acceptance by P. D’Ambra. (Qingcai Jiang and Zhenwei Cao contributed equally to this work.) (Corresponding authors: Wei Hu; Hong An; Jinlong Yang.)

Qingcai Jiang is with the University of Science and Technology of China, Hefei, Anhui 230026, China (e-mail: jqc@mail.ustc.edu.cn).

Zhenwei Cao is with the University of Science and Technology of China, Hefei, Anhui 230026, China, and also with Kuaishou Technology, Beijing 100085, China (e-mail: caozhenwei@kuaishou.com).

Junshi Chen and Hong An are with the University of Science and Technology of China, Hefei, Anhui 230026, China, and also with Laoshan Laboratory, Qingdao, Shandong 266237, China (e-mail: han@ustc.edu.cn).

Xinming Qin, Wei Hu, and Jinlong Yang are with the State Key Laboratory of Precision and Intelligent Chemistry, University of Science and Technology of China, Hefei, Anhui 230026, China (e-mail: whuustc@ustc.edu.cn; jlyang@ustc.edu.cn).

Digital Object Identifier 10.1109/TPDS.2025.3557621

I. INTRODUCTION

AMONG the major applications in high-performance computing (HPC), first-principles density functional theory (DFT) calculations hold significant importance for quantum mechanical simulations of materials [6]. DFT enables a direct connection between atomic structures and their associated physical properties, underpinning research in numerous scientific and engineering fields [22]. Given the critical role of DFT, extensive efforts have been made to develop efficient and accurate numerical approaches to solve the Kohn-Sham equations [2], [9], [20], [29], [34], [39], [45], [48]. Several numerical discretization methods exist for representing wavefunctions in DFT calculations, including numerical atomic orbital (NAO) basis sets [21], [25], [27] and Gaussian-type orbital (GTO) basis sets [13]. However, these localized orbital-based approaches are typically not systematically convergent for generic material systems. In contrast, the plane wave (PW) basis set [16], [18], [24] is widely favored due to its systematic convergence, accuracy, completeness, and orthogonality, making it particularly suitable even for strongly correlated systems. Consequently, PW-based DFT codes, such as the widely used commercial software VASP [19], [33] and the open-source package Quantum Espresso [15], [16], have gained extensive popularity in computational material science.

Recently, real-space discretization methods, such as SPARC [53] and DFT-FE [38], have demonstrated significant advantages in HPC environments due to their excellent parallel scalability and reduced communication overhead. These real-space methods avoid the global communication bottlenecks associated with the Fast Fourier Transform (FFT) operations required by PW methods. Nevertheless, the maturity, established software ecosystem, and robust convergence properties of PW-based methods continue to make them highly attractive and widely adopted in current research. In this work, we focus on the plane-wave-based PWDFT framework, aiming to exploit these advantages while enhancing computational efficiency and scalability.

Despite the popularity of plane wave basis set, the ultra high computation and memory cost of plane wave based DFT software remain a main stumbling block to the mainstream adoption of this highly accurate method. Conventional DFT calculations with plane wave basis set take $\mathcal{O}(N^3)$ floating point operations and cost $\mathcal{O}(N^2)$ memory footprint regarding the number of

electrons in the studied physical system N . For example, the number of basis N_{PW} in PW-based DFT calculations for a moderate-scale physical system with less than 5K atoms is approximately above 10^8 , which takes at least 10^{16} bytes of memory footprint and 10^{24} floating point operations. Therefore, it is hard for state-of-the-art PW-based DFT software to simulate large systems containing thousands of atoms even on modern supercomputers [3], [61]. As a result, exploring the physical properties of systems with thousands of atoms within PW-based DFT framework remains challenging.

Hence, a lot of research focuses on expanding the system size of PW-based DFT calculations on modern high-performance computing facilities [26], [30]. The new Sunway supercomputer is gaining popularity due to its ultra-high floating point performance (1.5 EFLOPS in total of the theoretical peak performance), which provides a promising way to push the limits of PW-based DFT calculations if we can fully leverage this supercomputer. However, there has been very little work on massively parallel PW-based DFT calculations, particularly on the new Sunway supercomputer, due to several challenges:

- Very limited 16 GB memory available for each process in Sunway supercomputer,
- Massive all-to-all memory accessing and communications in PW-based DFT software
- The complex programming scheme in Sunway supercomputer.

Plane Wave Density Functional Theory (PWDFT) [24] is one of the most scalable PW-based parallel DFT calculation frameworks to date. Based on our comprehensive characterization of PWDFT on the new Sunway supercomputer, we make two key observations: (1) A significant portion of the memory cost (e.g., temporary buffers and redundant pseudopotential information) in traditional PW-based DFT calculations can be alleviated with negligible communication cost. (2) Excessive small messages in PW-based DFT calculations incur heavy penalties due to traffic overload in communications and memory access.

Based on our observations, we revamp PWDFT on the new Sunway supercomputer in the following aspects: (1) We significantly reduce memory usage by efficiently refactoring the calculation and application of pseudopotentials with negligible communication cost. (2) We pack elements and use in-place data transformation to eliminate many temporary buffers and frequent small data transfers, leading to up to a 14x speedup. (3) We establish a multistage Allreduce operator, which leverages the characteristics of both hardware and the underlying communication library, enabling more comprehensive exploitation of network bandwidth performance. (4) We propose granularity-aware parallel mechanisms that consider the computation and communication patterns of different steps in PWDFT to select the most suitable parallel scale, further boosting the execution of PWDFT. (5) We establish a SWUC-assisted parallel programming framework to reduce the cumbersome programming handwork on the new Sunway platform. Additionally, we make some minor contributions to PWDFT, such as optimizing the data structure to make data access for CPEs more friendly and implementing a C++ library in PWDFT to monitor memory occupation at runtime.

The main contributions of this work can be summarized as follows:

- To address the key challenges of PW-based DFT code on the new Sunway supercomputer, we propose several optimization methods to leverage the limited 16 GB memory capacity of each process for massively parallel implementation of PWDFT code. Additionally, we devise several mechanisms to fully accelerate PWDFT.
- Through extensive numerical experiments, we demonstrate that our methods can achieve high scalability in both strong scaling and weak scaling. Furthermore, we achieve a speedup of 64.8 for a physical system containing 4,096 silicon atoms compared to the directly transported version, which is a very convincing result in PW-based DFT codes.
- We show that with our algorithms, along with parallel implementations and optimizations, we can study the graphene system with 16,384 atoms. This result greatly outperforms state-of-the-art PW-based DFT codes in terms of the physical system scale.

Most of the optimization methods and tools proposed in this work could extend to other PW-based DFT codes with the same computational characteristics, not only on the Sunway platform but also on other HPC facilities.

II. BACKGROUND

A. Background of PWDFT

PWDFT aims to solve an eigenvalue problem of this form:

$$HX = X\Lambda, \quad (1)$$

where H denotes the PW Hamiltonian, X presents the wavefunctions discretized with plane waves, and the eigenvalues of this problem λ are energies of the studied system.

Similar to many other DFT codes, PWDFT solves the eigenvalue problem in a self-consistent field (SCF) method. The SCF iterations in PWDFT contain two parts: (1) inner SCF for the eigenvalue problem of PW Hamiltonian and (2) outer SCF for checking the convergence of energy [42], as shown in Fig. 1. *At outer SCF*, the Kohn-Sham equation $H\Psi_i = \varepsilon_i\Psi_i$ is solved to obtain the improvement of the wavefunction Ψ_i for a given particle Hamiltonian H_i , and the wavefunction is expanded based on plane wave basis set. Outer SCF terminates when the output exchange energy V_{xc} agrees to input wavefunctions. *The inner SCF* is mainly responsible for diagonalizing the PW Hamiltonian by an iterative conjugate gradient eigensolver PPCG or LOBPCG. The inner SCF begins when the Hamiltonian matrix is constructed by wavefunctions, which aim to solve eigenproblems and acquire eigenvalues and eigenfunctions. Then the charge density $\rho(r)$ is calculated by eigenfunctions according to $\rho(r) = \sum_{j=1}^{N_e} |\psi_j|^2$, where ψ_j is the j -th eigenfunction and N_e represents the total valence charge number. Then exchange-correlation potential V_{xc} is calculated to be compared with the input for consistency. We present the major numerical kernels in each step on PWDFT on the right of Fig. 1 to enhance the readability.

In theory, wave functions are constructed using an infinite plane wave basis. However, E_{cut} is set to exclude waves with energy less than a certain input value. This cutoff energy determines the grid number in real space N_r according to $(N_r)_i = \sqrt{2E_{cut}} L_i / \pi$, where L_i is the length of the supercell in different dimensions (x , y and z). In our experiments, the default value of

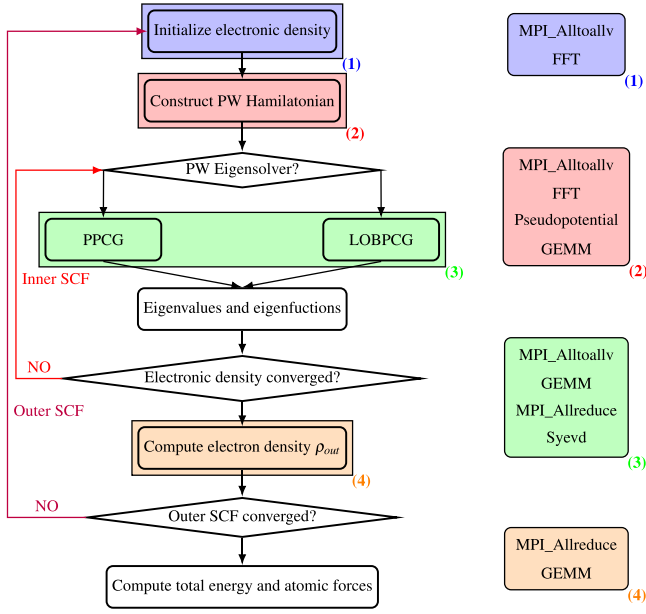


Fig. 1. Basic flowchart and 4 time-consuming steps with their corresponding numerical kernels (in the same background color) of PWDFT.

TABLE I
GRID NUMBER OF TYPICAL TEST CASES

System	x-axis	y-axis	z-axis	total
Si ₄₀₉₆	84	84	84	592704
Si ₈₁₉₂	88	88	176	1362944
G ₁₁₅₂₀	16	164	170	446080

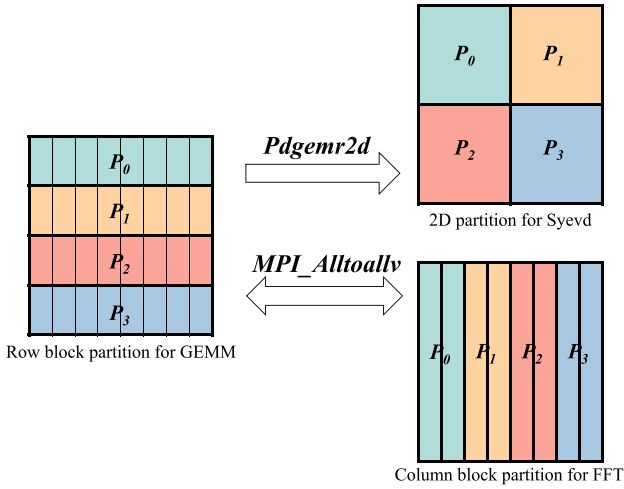


Fig. 2. Parallel data and task distribution schemes.

E_{cut} is set to 5 Hartree unless otherwise specified. Table I lists the grid number for certain systems in our experiments.

B. Parallelization Scheme

There are three main parallelization schemes as shown in Fig. 2 due to different intrinsic physical natures in PWDFT. The parallelization scheme is based on the distribution of the wavefunction matrix. *The first scheme* is the row block partition.

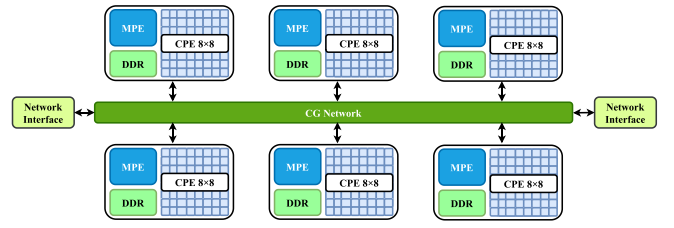


Fig. 3. Architecture of SW26010Pro processor.

This scheme facilitates matrix-matrix multiplications such as $S = \Psi^*(H\Psi)$ when $H\Psi$ is obtained. *The second scheme* is the column block partition. In this scheme, each process holds some entire wavefunctions, allowing a single process to perform a fast FFT without any data transformation or communication. This data distribution pattern also simplifies the calculation of $H\Psi$, where H is the Hamiltonian operator applied to wavefunctions. *The third scheme* is a 2-dimensional data partition used exclusively for Syevd, a ScaLAPACK function to acquire eigenvalues and eigenvectors. We perform MPI_Alltoallv and a ScaLAPACK routine Pdgemr2d to transform data patterns among the 3 schemes.

C. Problem Overview

In this section, we will elaborate on the three main obstacles that hinder the efficient parallelization of PWDFT on the new Sunway supercomputer.

1) *Limited Memory Size of Each Process:* Fig. 3 depicts the architecture of the SW26010Pro processor in the new Sunway supercomputer, comprising six core groups (CGs) each with one management processor element (MPE) and 64 computing processor elements (CPEs). Each processor is equipped with 96 GB of memory, evenly allocated at 16 GB per CG. The new Sunway supercomputer employs two memory partition schemes: (1) the common scheme, where each core group acts as a separate processor with 16 GB of memory, and (2) the large memory scheme, where six core groups in a process share 96 GB of memory and act as a single process. However, numerical libraries that are extensively used in PWDFT like FFTW [12], BLAS [11], and LAPACK [1] are tailored to the common scheme, implying that each process of PW-based DFT can only use the common memory partition scheme, which allocates a very limited 16 GB memory footprint per process.

2) *Massive All-to-All Data Communications:* The predecessors of the new Sunway, specifically the Sunway Taihu Light, had a peak memory bandwidth of 136 GB/s, creating a significant bottleneck for various applications. In contrast, the new Sunway boasts a bandwidth transfer from global memory to the LDM of CPE around 307 GB/s via direct memory access (DMA). The bandwidth among the CPEs within the CPE cluster, via remote memory access (RMA), has a theoretical peak performance of 460 GB/s [62]. However, data sizes per transfer must exceed 1024 and 256 bytes to achieve 80% bandwidth utilization for DMA and RMA, respectively.

PW-based DFT calculations require extensive all-to-all data communications due to the intrinsic physical properties of the huge dense Hamiltonian matrix. These communications remain costly even on the new Sunway supercomputer, despite specialized hardware and software support. Additionally, the transformation of the parallelization scheme involves substantial small-volume and irregular memory access to ensure data continuity. These memory accesses are time-consuming due to their small granularity and inadequate data locality, hindering the full utilization of the cache hierarchy. Consequently, heavy all-to-all data communications can severely degrade performance in PW-based DFT calculations.

3) *Complicated Programming Framework*: The major computing resource of the new Sunway supercomputer is provided by the 64 CPEs of each core group. To effectively leverage their significant floating-point performance, programmers usually need to manually orchestrate the CPEs through a fine-grained Athread programming interface. This interface is an effective but cumbersome thread library provided by the vendor.

III. MECHANISMS

In this section, we introduce the mechanisms we propose to address the aforementioned challenges described in Section II-C.

A. Refactor of the Pseudopotential Calculations

Pseudopotentials are commonly used in DFT calculations to approximate the atomic nucleus and inner-shell electrons as ion entities, a practice that is justified given that chemical properties are primarily influenced by the behavior of the outermost valence electrons [37], [54].

In Plane Wave Density Functional Theory (PWDFT) software, each atom is represented by specific pseudopotential information that adjusts each wave function accordingly. Originally, each computing process maintained the entire dataset of pseudopotentials. While the memory required for a single atom's pseudopotential is not substantial, the total memory consumed by all atoms' pseudopotentials significantly limits PWDFT's ability to model larger systems. This limitation is particularly acute on the new Sunway system, which has a restricted memory capacity of 16 GB. To address this, we have redesigned the pseudopotential calculations by implementing a distributed storage scheme to reduce the memory bottleneck.

The original procedure of applying pseudopotential to wave functions is illustrated in Fig. 4(a). These functions are distributed across various processes. For each wave function stored locally, the algorithm sequentially processes every atom, adjusting the wave function according to the corresponding pseudopotential. When the pseudopotential is distributed across processes, each process retains only a portion of the overall pseudopotential data. Therefore, to apply the pseudopotential, it has to be retrieved from other processes through a network interface. In the case of a basic algorithm, this necessitates the redundant transmission of the same atomic pseudopotential across distinct wave functions.

Therefore, we apply the pseudopotential of an atom to all local wave functions once obtained, thereby eliminating redundant

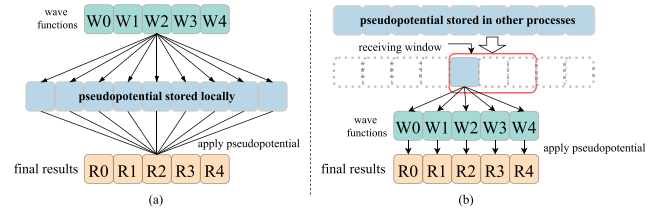


Fig. 4. Refactor of pseudopotential process: (a) naive implementation where each process holds the entire pseudopotential, (b) the refactored approach where each process holds a portion of pseudopotential, in which the dotted, uncolored blocks indicate the absence of pseudopotential in the process.

transmission of pseudopotential. Additionally, we employ a sliding window algorithm to address significant communication overhead, enabling overlapping computation and communication. This approach, illustrated in Fig. 4(b), allows a process to receive the upcoming pseudopotential within the sliding window while applying the current pseudopotential to the local wave functions. The communication overhead can be fully overlapped with the flexible window size.

This optimization significantly reduces memory usage. For instance, in a graphene system containing 11,520 atoms, the pseudopotential of a single atom is roughly 0.3 MB when $E_{cut} = 10$. In the naive implementation, each process must store the pseudopotential of all atoms, leading to an estimated memory footprint of approximately 3.4 GB. This constitutes about 21% of the entire memory per process. However, once we refactor the pseudopotential process, the memory usage becomes insignificant.

B. Memory Accessing Optimization

1) *The Mapping and Buffer Matrix*: As mentioned, the parallelization scheme that involves both row and column partitioning requires global communication through MPI_Alltoallv, as illustrated in Fig. 2. However, this global communication via MPI_Alltoallv in PWDFT to transform data between row and column partitions introduces data transformation to *make the data contiguous in memory*.

For simplicity, we take the data transformation from column partition to row partition as an example. The data transformation in the naive implementation of PWDFT involves two mapping matrices and buffers, one before and one after the MPI_Alltoallv operation. We illustrate the data transformation formula in Fig. 6, where blocks of the same color are sent to the same processor, and the number in each block of the original data and buffer represents each data element. For example, the original data matrix in Fig. 6 shows the column partition data in one of the four processes, which is sent to four processes, indicated by different colors. In the column-major format, the data block in light turquoise is not contiguous. Thus, the block containing “10.0” in the original data matrix is mapped to the 3rd position (starting from 0) in the buffer matrix in Fig. 6 for MPI_Alltoallv.

2) *Optimizations*: Here, we design an in-place data transformation, as illustrated in Fig. 5, to reduce the heavy memory occupation of the buffer.

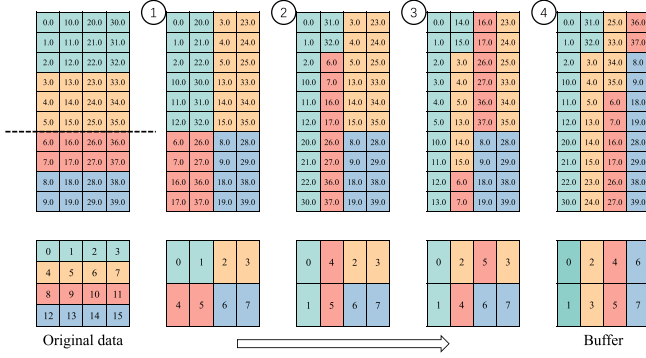


Fig. 5. An example of the in-place data transformation. The circled numbers denote the procedures in the transformation.

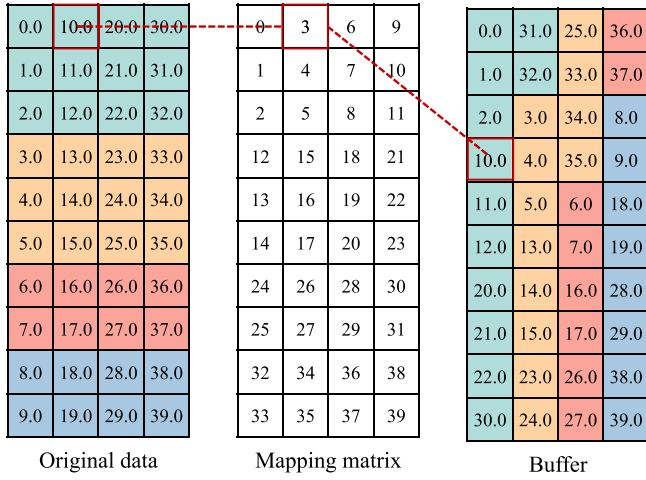


Fig. 6. An example of the mapping matrix in the data transformation.

We combine the data in the same column of each process into *sub-blocks*, as shown at the bottom of Fig. 5. The number in each sub-block indicates the target position for the transformation. This sub-block method offers three benefits: (1) it increases the granularity of data transfer, (2) it significantly decreases the cost of the mapping matrix and (3) it enhances the temporal and spatial locality of the associated data transfer.

Consider a case where the number of processes is P , the number of rows and columns in the wavefunction matrix is r and c , and $BlockSize = \lfloor r/P \rfloor$. In this case, $re \equiv r \bmod P$, and there are $(re * c)$ sub-blocks with the size of $(BlockSize + 1)$ and $[(P - re) * c]$ sub-blocks with the size of $BlockSize$. Fig. 5 shows an example with $r = 10$, $c = 4$, $P = 4$, and $BlockSize = 2$.

Our method starts by dividing the column-partition matrix into two sections as indicated by the dashed line in Fig. 5, ensuring that sub-blocks within each region are of uniform size. We then exchange sub-blocks within the same part to create larger sub-blocks, further *increasing the granularity of computation*, which is stated in Step ① of Fig. 5. The exchange procedure is straightforward since the target position of each sub-block is reachable to make them sequential in the region. Following that, we reposition the enlarged sub-blocks to ensure

their continuity across the entire region, as demonstrated in Steps ②, ③, and ④ of Fig. 5.

In this sub-block transformation scheme, storing the whole mapping matrix is highly redundant, instead, we generate the sub-block mapping index on-the-fly. In our experiments, we observe that the computing pattern of this mapping is highly regular, which means we can distribute the mapping workload to each CPE almost evenly. As a result, we eliminate the mapping matrix without sacrificing any performance.

After MPI_Alltoallv, each process holds specific rows of the entire matrix. We then perform the in-place column rearrangement, which also requires mapping and buffer matrices in the original version.

3) *Performance and Analysis*: In the column partition, each process holds a matrix with r rows and c columns. Before MPI_Alltoallv, the mapping matrix and the temporary buffer have the same size. Each element in the matrix is represented using double-precision floating-point format, which requires 8 bytes per element. Therefore, the memory requirement for the buffer is $8rc$ bytes. Regarding the mapping matrix, it can be stored in an integer format, which occupies $4rc$ bytes in memory. Consequently, the mapping matrices and buffers occupy a total memory of approximately $24rc$ bytes. For the silicon system with 4096 atoms (Si_{4096}), which contains $4096 \times 2 = 8192$ wavefunctions (each silicon atom has four valence electrons, and two valence electrons correspond to a wavefunction), in this case, when using 64 computational grids (CGs) to run the simulation, we have:

$$c = \frac{8192}{64} = 128 \quad \text{and} \quad r = 592704.$$

The memory size saved by our aforementioned methods is:

$$\frac{592704 \times 128 \times 24}{1024 \times 1024} \approx 1736 \text{ MB}.$$

Meanwhile, our optimization also improves the efficiency of data transformation as the utilization of memory bandwidth is enhanced by transferring data at the sub-block granularity. For example, when 1024 CGs are used to perform data transformation on Si_{4096} , we achieve a 14.2x speedup compared with the original version.

C. Communication Optimization

It has been documented that MPI collective communication (e.g., MPI_Allreduce, MPI_Bcast, etc.) experiences *performance fluctuations* on some supercomputers [30]. In other words, the performance of network bandwidth varies with the volume of data being transferred. A similar phenomenon has been observed on the new Sunway supercomputer, as depicted in Fig. 7. Additionally, a critical observation derived from our experiments suggests that the performance of MPI_Allreduce on the new Sunway supercomputer is markedly *superior when the number of participating processes is an exponential power of 2*.

Therefore, we propose a multistage Allreduce to mitigate network performance fluctuations and optimally leverage the

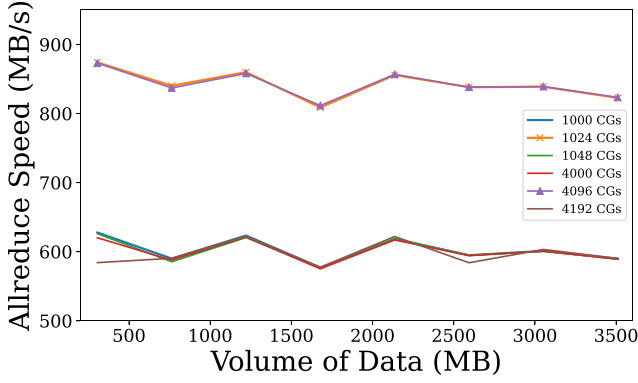


Fig. 7. Performance of MPI_Allreduce regarding different communication volumes on the Sunway supercomputer.

Algorithm 1: Algorithm of Multistage Allreduce.

```

1: rank: mpi rank in the global communication domain
2: P: number of processes in the global domain
3: C: parameter, each row domain has C or C + 1 processes
4:  $m \leftarrow P \bmod C$ 
5:  $n \leftarrow \text{rank} * (C + 1)$ 
6: if rank < n then
7:    $i \leftarrow \lfloor \text{rank} / (C + 1) \rfloor$ 
8:    $j \leftarrow \text{rank} \bmod (C + 1)$ 
9: else
10:   $i \leftarrow \lfloor (\text{rank} - n) / C \rfloor + m$ 
11:   $j \leftarrow (\text{rank} - n) \bmod C$ 
12: end if
13: Divide the matrix into C blocks, named  $B_0, B_1 \dots B_C$ 
14: for  $k \in [0, C)$  do ▷ Stage ①
15:   Process with  $j = k$  reduce block  $B_k$  from row domain
16: end for
17: if  $k \in [0, C)$  then ▷ Stage ②
18:   Execute MPI_Allreduce in column domain for Block  $B_k$ 
19: end if
20: for processes in the same row do ▷ Stage ③
21:   Exchange data block with the row domain processes to
      obtain the entire matrix.
22: end for

```

communication characteristics of the new Sunway supercomputer.

1) *Notation and Prerequisite*: We use a predefined parameter *C* and arrange the processes into a two-dimensional grid, wherein processes in the same row are segmented into a row domain, and likewise, processes in the same column are segmented into a column domain. For each process, a pair of *i, j* indicates that the process is situated in the *i* - *th* row domain and the *j* - *th* column domain. The entire matrix is partitioned into *C* blocks, named $B_0, B_1, \dots, B_{C-1}$. These *C* blocks correspond to the first *C* processes (having ranks from 0 to *C*-1) within the row domain.

2) *Algorithm and Implementation*: In stage ①, as illustrated in lines 14 to 16 of Algorithm 1, processes within the same

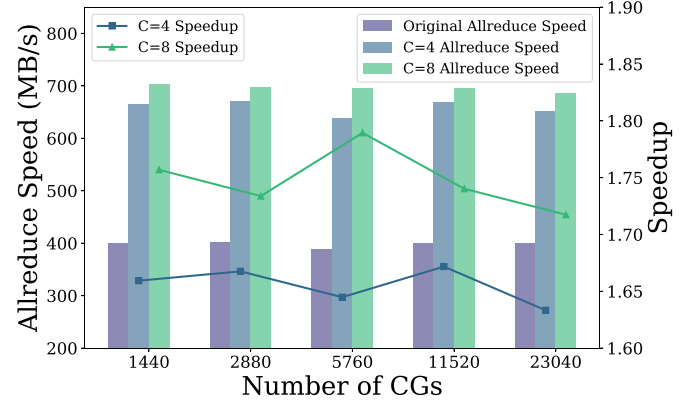


Fig. 8. Allreduce performance speedup with the same communication volume and different processes.

row domain execute MPI_Reduce *C* times to reduce the corresponding block, processing one block per iteration. Consequently, the process with rank 0 in the row domain obtains the reduced result of block B_0 , while the process with rank 1 obtains the reduced result of block B_1 , continuing in this manner for subsequent processes. It can be observed that for the processes in the row domain, each retains a portion of the reduced result of the matrix (except for extra processes where $j \geq C$). In other words, $B_0, B_1, \dots, B_{C-1}$ together hold the reduced result of the entire row domain. As for extra processes, they just send data to other processes with $j < C$ at each MPI_Reduce, and they don't need to receive any data. In stage ③ (lines 17 to 19 of Algorithm 1), processes within the same column domain *j* (i.e., processes with identical *j* values) all retain the same block B_j ($j \geq 0$ and $j < C$). An MPI_Allreduce operation is then performed over the column domain, ensuring that every process within column domain *j* holds the complete reduced result for block B_j . For extra processes with $j < C$, they are not involved in this stage. In stage ③ (lines 20 to 22 of Algorithm 1), it is evident that the combination of blocks $B_0, B_1, \dots, B_{C-1}$ represents the entire MPI_Allreduce result. Therefore, at this stage, data is exchanged within the row domain via MPI_Alltoallv.

This approach takes full advantage of the fact that the performance of the MPI collective communication is significantly better when the number of participating processes is an exponential power of 2.

We remark that although extra processes (with $j \geq C$) obtain slightly less communication workload, there will be only $P \bmod C$ row domains containing an extra process (given that $P \gg C * C$). And our numerical results demonstrate a satisfactory workload balance.

3) *Performance and Analysis*: We evaluated our multistage method in the simulation of a graphene system with 11,520 atoms using processes of different sizes. The results in Fig. 8 show that our implementation is 63.3% to 67.1% faster when *C* = 4 and 71.7% to 78.9% faster when *C* = 8. To better illustrate the benefits of our optimization, we also evaluated the performance of our algorithm with different communication volumes using the same 4000 processes. As shown in Fig. 9, the

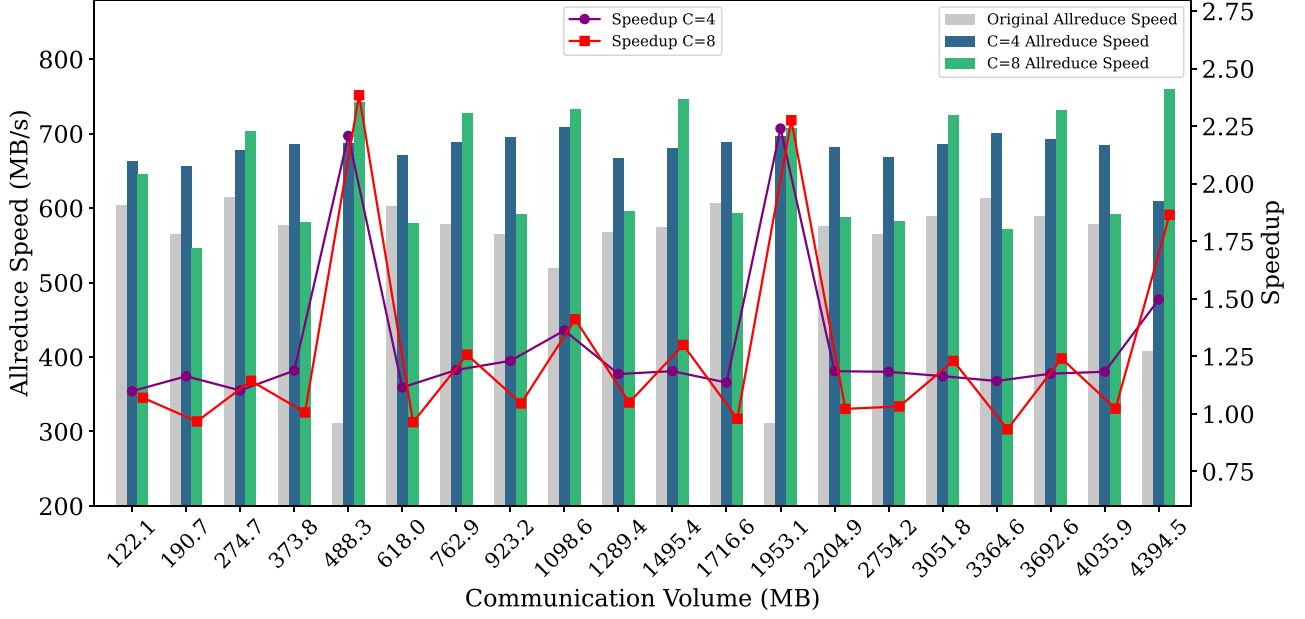


Fig. 9. Allreduce performance speedup with different communication volume and the same processes.

TABLE II
SYEVD TIME FOR Si_{8192} WITH DIFFERENT PROCESSES

ProcNum	1000	1024	2048	2056	4096	4112
Time (s)	698	210	439	21349	619	9858

stepwise communication reduction algorithm introduced in this manuscript exhibits enhanced performance in the vast majority of instances when $C = 4$, achieving an acceleration of up to approximately 2.5 times under optimal conditions. Although the performance gains at $C = 8$ are comparatively modest, they still surpass those of the baseline approach in nearly all cases.

We emphasize that our proposed method is highly adaptable, accommodating any number of processes, and exhibits superior performance when the number of processes is a power of 2.

D. Granularity-Aware Parallelization

One of the most time-consuming procedures in PWDFT is computing eigenvalues and eigenvectors using Syevd in ScaLAPACK. The performance of ScaLAPACK is governed by two factors: (1) the peak performance for computing on each process and (2) the network communication bandwidth. It has also been reported that the scalability of ScaLAPACK is limited on modern supercomputers because the workload partitioned in each process is scarce and frequent, and time-consuming communication dominates the performance [2]. We evaluated the Syevd implementation for Si_{8192} using different numbers of processes on the new Sunway supercomputer. The results presented in Table II reveal that the time to solution may even increase as the number of processes grows. In this case, Syevd performs best when the number of involved processes equals 1024.

Therefore, adopting all processes to perform Syevd may suffer performance degradation, and it's sufficient to improve it by simply adopting fewer processes. To address this problem, we propose a granularity-aware parallelization scheme, which enables PWDFT to run with an optimal process number during different execution steps. The optimal number of processes is determined by considering various factors, such as computational intensity, communication volume, hardware computational capacity, and network bandwidth available on the new Sunway supercomputer.

As discussed in Section III-C, MPI communication benefits from network topology and MPI implementation when the number of processes is a power of 2. This effect is even more pronounced in the Syevd routine, as we observe significant time-to-solution changes when the number of processes changes from 1000 to 1024, 2048 to 2056, and 4096 to 4112, as shown in Table II. Thus, there is a minimum threshold for the number of processes for a given atomic system, noted as P_{min} . We have developed a C++ library for memory analysis of PWDFT (discussed in Section III-F), which allows us to analyze memory usage and determine the threshold for the number of processes. For a silicon atomic system, one atom corresponds to two wavefunctions, the minimum granularity for FFT and matrix multiplication. Therefore, the maximum number of processes is twice the number of atoms (noted as A).

The optimal number of processes (noted as P_{opt}) for Syevd falls within the following range: $P_{opt} \in [P_{min}, 2 * A] \wedge P_{opt} = 2^k, k \in \mathbb{N}^+$. Based on this, we can determine the optimal process number from the candidates by balancing computing intensity and communication costs. For other parts of the PWDFT, such as the calculation of the pseudopotential, the increasing number of processes leads to a shorter time to solution. Within the range of $[P_{min}, 2 * A]$, more processes indicate faster execution.

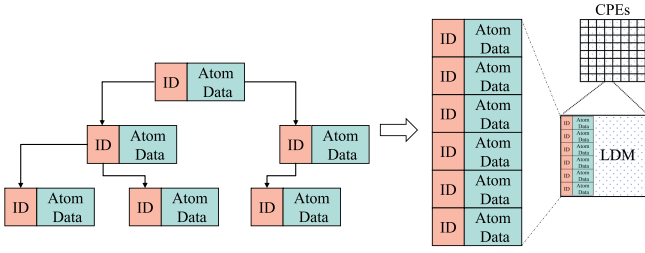


Fig. 10. Data structure optimization of pseudopotential.

E. SWUC-Assisted Heterogeneous Multi-Core Programming

The extensive floating-point performance on the new Sunway supercomputer is mainly provided by CPEs. Each CPE has a very limited (256KB) scratch-pad memory (SPM) that can be configured as local data memory (LDM) or Local Data Cache (LD Cache). However, effectively harnessing the CPEs to maximize the acceleration of PWDFT proves to be a challenging task. It requires writing extensive code via the Athread library and working within the confines of the limited LDM.

Fortunately, a new programming framework called ShenWei Universal C/C++ (SWUC) [5] has been proposed to simplify programming with CPEs. Thanks to several new attributes and compiler directives, writing code running on CPEs and MPE can be simply achieved by calling lambda expressions, in which we divide data for each CPE and specify the code to be executed. We adopt the framework in the calculation of pseudopotential, wavefunctions, ion density, and energy. We divide the data according to the layout of CPEs, batch data, and transfer it to the LDM of CPEs, and obtain the final results via a lambda function.

However, for some more complex computing scenarios, SWUC is not applicable yet. In such cases, we have to use the Athread library, the most primitive interface to use CPEs, to accelerate PWDFT. The numerical result of the SWUPC-assisted CPE programming will be discussed in Section IV-D.

F. Other Optimization and Tool

In this subsection, we describe briefly our minor contributions to PWDFT on the new Sunway supercomputer.

Data Structure Optimization: When constructing the PW Hamiltonian, it's necessary to look up the pseudopotential for a given atom. However, the original implementation of PWDFT, which targets general-purpose CPUs, used `std::map` from the C++ Standard Template Library to perform the look-up. This method suffers from poor performance due to its indirect access, especially when the system size increases. Additionally, the CPEs on SW26010Pro are unable to allocate from the main memory shared across cores, making `std::map` inaccessible on CPEs and hindering pseudopotential lookup acceleration. Considering that there are typically no more than 200 different types of atoms, to address these issues, we replace `std::map` with a small array. By indexing the array with the atom number, we are able to acquire the pseudopotential with one direct access, resulting in significant performance improvements, as shown in Fig. 10. This new mechanism is also more CPE-friendly, as the LD Cache makes loading the pseudopotential efficient in the

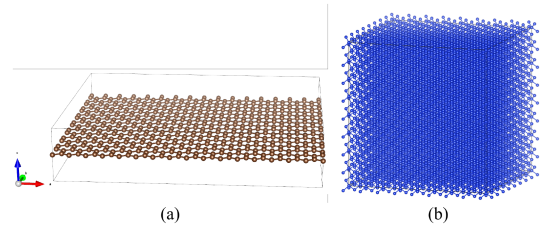


Fig. 11. Testing systems: (a) graphene system and (b) bulk silicon system.

case of direct indexing. Compared to the previous design, this method boosts the performance of the PW Hamiltonian step by a factor of 10.

Runtime Memory Usage Monitor: To obtain a more detailed understanding of memory usage, we implemented a runtime memory monitor library for PWDFT. Memory usage can be categorized into three main types: global memory space for global variables, temporary memory space allocated for temporary variables in a function call, and memory space to store DFT information. We ignore the first type, as it is relatively small in PWDFT. We do not need to pay special attention to the second type of memory usage if no memory bound occurs in the function. We consider all variables for DFT information and calculate the used memory size. For memory-bound functions, we enumerate the large temporary variables that occupy significant memory space and determine the total size of the occupied space, including DFT information. The memory usage information is outputted whenever there is a substantial change in memory usage.

G. Portability

Our optimizations can significantly accelerate PWDFT and extend the limits for DFT calculation. The techniques in this work can be readily applied to other platforms, except for the CPE optimizations. Additionally, these optimizations are relevant and serve as a reference for DFT software across various platforms. Specifically, the refactoring of the pseudopotential algorithm and memory access optimization can be easily ported to other DFT software on platforms such as GPU or many-core X86/Arm CPU supercomputers without significant code changes. Furthermore, communication optimization and the granularity-aware parallelization scheme can be applied to other software on the new Sunway supercomputer and other computers with similar characteristics [2], [30]. Our optimization with CPE in Section III-E is specific to the Sunway platform and is highly beneficial for other research conducted on the Sunway supercomputer.

IV. NUMERICAL RESULTS AND ANALYSIS

A. Setup of the Test Physical Systems and Testing Environment

The testing systems include two parts: graphene (labeled as “G”) and cubic silicon systems (labeled as “Si”), as shown in Fig. 11. These systems are denoted respectively as G_{11520} , G_{16384} , Si_{64} , Si_{216} , Si_{512} , Si_{1000} , Si_{4096} , and Si_{8192} . The corner marks indicate the number of atoms in each system. Notably, our

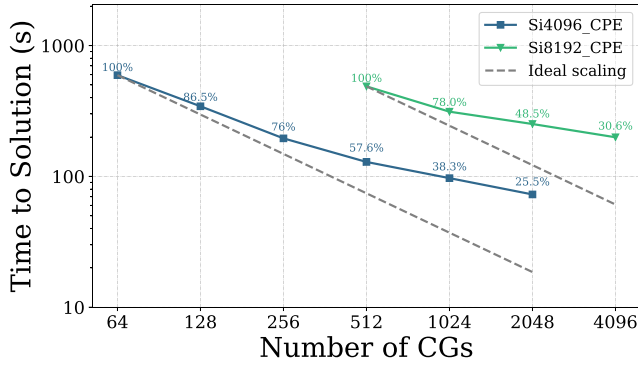


Fig. 12. Strong scaling of PWDFT.

parallelization approach is well-suited for handling large-scale quantum simulations, making it applicable to other complex physical systems such as amorphous silicon [43], [52].

In our experiment, the kinetic energy cutoff E_{cut} is set to 5 Hartree unless otherwise specified. This value has been found sufficient for achieving convergence in silicon crystal calculations [4]. We use the same E_{cut} in graphene systems for consistency. Our optimization approach has good effects for different values of E_{cut} . All numerical tests are performed on the new Sunway supercomputer, and each MPI process is applied to one CG (including 1 MPE and 64 CPEs) with 16GB of memory. We consider the time per outer SCF iteration as the time to solution.

B. Strong Scalability

To evaluate the strong scalability of Si₄₀₉₆ and Si₈₁₉₂, we use the parallel efficiency defined as

$$\text{efficiency} = \frac{\text{Speedup}}{\text{Multiple of CGs}}.$$

The speedup is compared with the minimum number of CGs (64 CGs for Si₄₀₉₆ and 512 CGs for Si₈₁₉₂) capable of performing the calculation. We use 64 to 2048 CGs when simulating Si₄₀₉₆ to test the strong scalability of PWDFT and 512 to 4096 CGs for Si₈₁₉₂. The results are shown in Fig. 12. From the figure, we can see that, when scaling up to 2048 CGs (Si₈₁₉₂), PWDFT maintains a parallel efficiency of 48.5%, which is quite acceptable for PW-based DFT calculations. However, the parallel efficiency is not satisfactory when scaling up to half the number of atoms. Even so, we must note that such mediocre parallel efficiency is already much better than the original version and other PW-based DFT software. In Section III-D, due to the poor scalability of Syevd, the optimal process number is always selected in our granularity-aware parallelism for Syevd in the range of $P_{opt} \in [P_{min}, 2 * A] \wedge P_{opt} = 2^k, k \in \mathbb{N}^+$. Hence, when the number of processes reaches a sufficient threshold to achieve the best performance for Syevd, it remains unchanged. Considering that Syevd often occupies a significant portion of the total computation time, it is reasonable to conclude that PWDFT does not exhibit excellent strong scalability and parallel efficiency. Additionally, massive global communication, as discussed in Section II-C2, also partially contributes to this inefficiency.

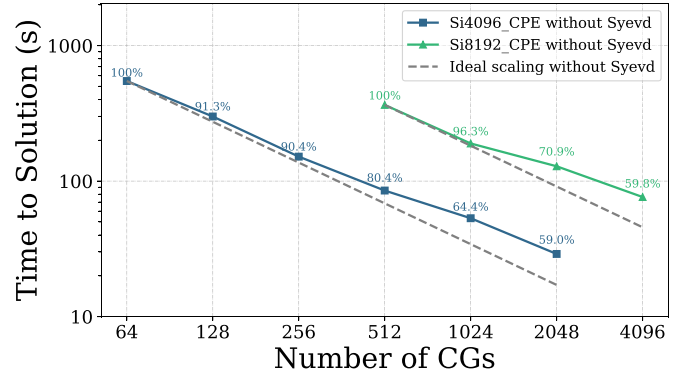


Fig. 13. Strong scaling of PWDFT without syevd.

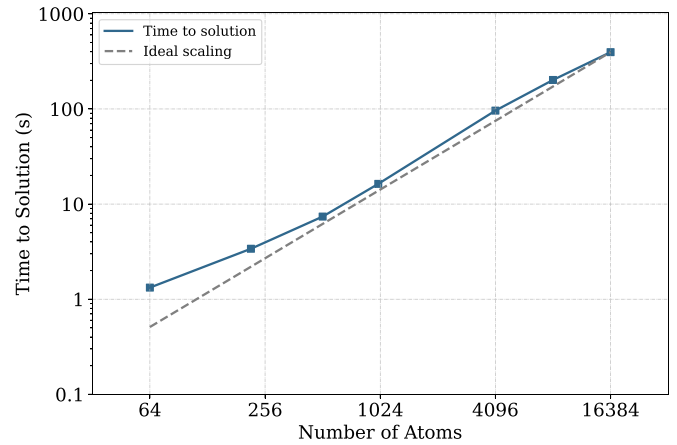


Fig. 14. Weak Scaling of PWDFT w.r.t. the number of electrons.

As is shown in Fig. 13, the result of the above systems excluding Syevd time shows a much better and reasonable parallel efficiency, as over 59% of the parallel efficiency is achieved when the number of CGs reaches half of the atom's number. Moreover, under such circumstances, massive global communication starts to play a dominant role when the number of CGs is greater than half of the atom's number, which prevents the code from scaling further.

C. Weak Scalability

Our method offers a significant reduction in memory cost during the calculation steps of PWDFT. As a result, we can efficiently study a much larger physical system using far fewer computing resources. In our testing of weak scalability for Si₆₄, Si₂₁₆, Si₅₁₂, Si₁₀₀₀, Si₄₀₉₆ and G₁₆₃₈₄, we utilize one-quarter of the atom number as the process number. This approach enables us to optimize computational efficiency while maintaining high accuracy in our calculations.

The weak scaling of PWDFT for simulating systems consisting of 64 to 16,384 atoms is shown in Fig. 14, from which we can see that PWDFT exhibits excellent weak scaling properties. Since the computational complexity of PWDFT is $O(N^3)$, and the number of CGs is set to a quarter of atoms number, the ideal scaling should be $O(N^2)$, which is represented by a dotted line in Fig. 14.

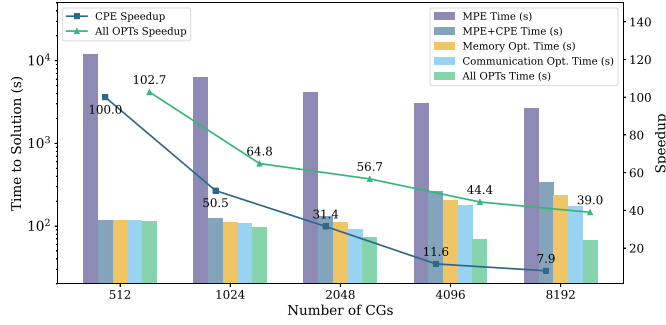
Fig. 15. Speedup for Si_{4096} with master version.

TABLE III
MINIMUM CGs REQUIRED TO CALCULATE THE SYSTEM

System	Master Version	Optimized Version
Si_{1000}	512	25
Si_{4096}	512	64
Si_{8192}	out of memory	512
G_{11520}	out of memory	720
G_{16384}	out of memory	2048

As shown in Fig. 14, in the case of small systems, computing resources are not the dominant factor, and the cost of communication becomes relatively more significant, making scalability a challenge. Therefore, it is difficult to ignore the impact of communication costs, which can limit the ability of the system to scale effectively. When the atom's number is greater than 1000, the actual execution time is very close to the ideal scaling, which suits our computational complexity well. Here for G_{16384} , the simulation with 4,096 CGs is only 395 seconds, which makes DFT calculation for large systems more practical.

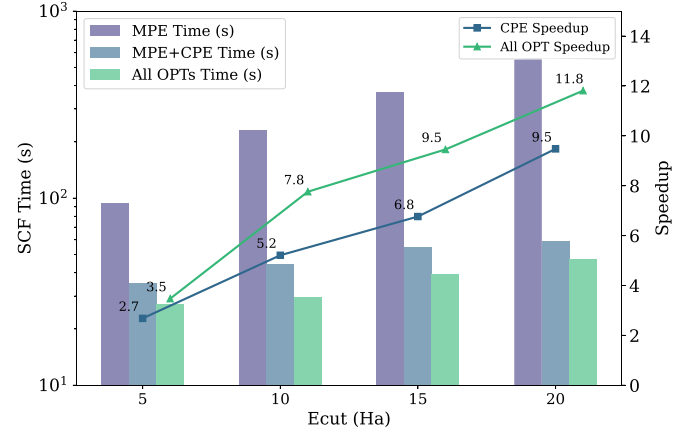
D. Speedup Compared With MasterVersion

In Fig. 15, we compare the acceleration effects of using CPE alone and employing all optimization techniques in the simulation of Si_{4096} . We use the master version's results, which are directly transferred to the Sunway system and rely solely on MPE, as the baseline for our comparison.

The results show that when only adopting CPE to accelerate, the speedup varies from 100.0 to 7.9, and decreases dramatically with the increase of the number of CGs. For comparison, when utilizing all the mechanisms proposed in Section III, the acceleration effect is noticeably enhanced. Under more common circumstances, the fully optimized version achieves 64.8 times faster than the master version when we adopt one-quarter of the atom number, i.e., 1,024 CGs.

For both versions, the acceleration decreases with an increase in CGs. This is because an increase in CGs indicates less computational workload, and the cost of communication also rises.

Meanwhile, memory optimization in Sections III-A and III-B significantly extends the limit of the studied system for PWDFT, as shown in Table III. The master version requires at least 512 CGs to calculate a silicon system containing 4,096 atoms

Fig. 16. SCF iteration time and speedup comparisons at different E_{cut} values.

(Si_{4096}). However, after our optimization, only 64 CGs are needed. Furthermore, only 512 CGs are required for a silicon system with 8,192 atoms (Si_{8192}), which is quite impressive for PW-based DFT calculations. Consequently, the limit of PWDFT is extended from a physical system of 4,096 atoms to one encompassing 16,384 atoms.

E. Speedup With Different Energy Cutoff

To comprehensively examine the performance of our optimization strategies across various computational conditions, we conducted additional tests by varying the plane-wave energy cutoff (E_{cut}). We selected four different E_{cut} values (5, 10, 15, and 20 Ha), representing both our default setting (5 Ha) and more realistic values typically encountered in practical DFT calculations. We compared the SCF iteration times of three versions of our implementation: the Master version (MPE, utilizing only the main processing element), the version with additional computing processing elements (MPE+CPE, including slave cores), and the fully optimized version (All OPT), incorporating all optimization techniques proposed in this work. The results are summarized in Fig. 16.

From Fig. 16, we observe the following:

- At all tested energy cutoffs, both the MPE+CPE and All OPT versions significantly outperform the Master version, clearly demonstrating that utilizing slave cores effectively accelerates SCF calculations.
- The speedup of the MPE+CPE version relative to the Master version improves as the energy cutoff increases, achieving approximately $2.7\times$, $5.2\times$, $6.8\times$, and $9.5\times$ at 5, 10, 15, and 20 Ha, respectively. This trend underscores the effectiveness of parallelization using additional computing elements as computational intensity increases.
- More importantly, the fully optimized All OPT version, which includes all optimizations discussed in this paper, consistently delivers superior performance. Specifically, it achieves a speedup of approximately $3.5\times$ at 5 Ha, which steadily rises to approximately $7.8\times$, $9.5\times$, and $11.8\times$ at 10, 15, and 20 Ha, respectively. This substantial improvement further validates the robustness and practical applicability of our proposed optimization strategies.

These results confirm that the optimization methods proposed in this study are effective across a broad range of energy cut-offs, ensuring that our conclusions remain valid and relevant under realistic computational scenarios commonly encountered in large-scale PW-based DFT simulations.

V. DISCUSSION

One of the hot spots in PWDFT is solving the eigenproblem distributively by calling SyeVd. While other dominant functions have been optimized and parallelized, the SyeVd function does not benefit significantly from an increase in the number of processes due to communication constraints. This has been a consistent finding in our research.

In terms of computational ability, PWDFT is still memory-bound for the simulation of larger systems on the new Sunway supercomputer. Memory bottleneck occurs in the storage of pseudopotential and Hamiltonian matrices. To our knowledge, our work has already reached the boundary of PW-based DFT on the new Sunway supercomputer. To further expand the capabilities of PW-based DFT software on the new Sunway supercomputer, one must either (1) create novel algorithms with reduced memory complexity or (2) develop numerical libraries that align with the large memory scheme of the new Sunway supercomputer. It is important to emphasize that this work does not primarily focus on these two aspects.

Our memory optimization approach reduces random memory access, thereby mitigating the negative impact of data movement and making PWDFT more adaptable to new Sunway's ultra-high computing-to-memory ratio architecture.

A. Potential Applications of PWDFT-SW

The optimization techniques presented in this work are primarily developed and evaluated for the Sunway supercomputer; nevertheless, many of these strategies are not inherently limited to the Sunway architecture and can be generalized to other state-of-the-art supercomputing platforms, including GPUs [40], [41], Fugaku [44], and Summit [51].

For instance, the memory accessing optimizations and pseudopotential refactoring schemes, designed to reduce redundant communication and enhance data locality, are directly applicable to GPU-based platforms such as Summit, which benefit significantly from efficient memory management and reduced communication overhead. Similarly, the granularity-aware parallelization method can be adapted for other many-core CPU architectures such as Fugaku, where appropriate resource allocation and optimized communication patterns can greatly enhance computational performance.

Future work will involve porting and tailoring these optimization strategies explicitly for GPU-based and ARM-based supercomputers. Such efforts would enable detailed benchmarking across different architectures, highlighting the universality and adaptability of our proposed optimizations. Additionally, further investigation into platform-specific tuning and potential new bottlenecks on these architectures would provide valuable insights, thereby broadening the applicability and impact of our methods within the high-performance computing community.

B. Integration With Machine Learning Methods

Machine learning (ML) techniques have demonstrated significant potential in accelerating and enhancing DFT calculations. Recent prominent examples include DeepH [35], [50], which employs graph-based neural networks to predict DFT Hamiltonians directly from atomic structures, thereby circumventing expensive self-consistent iterations. Another noteworthy development is DeepKS [7], [8], which uses neural-network corrections to improve the accuracy of conventional DFT exchange-correlation functionals by learning from high-accuracy reference data. Additionally, DeepMD [47], [55] has revolutionized *ab initio* molecular dynamics simulations by using ML-driven interatomic potentials, significantly enhancing both computational efficiency and scalability. The remarkable success of DeepMD, highlighted by the ACM Gordon Bell Prize [36], underscores the transformative impact of ML in enabling large-scale molecular dynamics simulations with DFT-level accuracy. Furthermore, recent generative models such as MatterGen [56] have also demonstrated promising capability in efficiently generating stable inorganic materials, potentially complementing ML-accelerated DFT calculations. Additionally, the recent N2AMD framework [57], employing E(3)-equivariant neural networks to predict DFT Hamiltonians for nonadiabatic molecular dynamics, offers a promising direction for further accelerating PWDFT-based simulations of large-scale materials.

Beyond these specific frameworks, general ML strategies like Behler-Parrinello neural networks [17] and Gaussian Approximation Potentials (GAP) [32] have also been integrated successfully with DFT to create computationally efficient surrogate models, reducing computational demands while preserving high fidelity. Integrating such ML-based methods with the optimization strategies proposed in this study holds substantial promise. Future investigations could involve combining our parallel optimization methods with ML-based approximations to further enhance efficiency, particularly in large-scale materials simulations. Such integration could expand the application of PWDFT to even larger and more complex systems, offering avenues for significant future research.

VI. RELATED WORKS

A. Other Parallel PW-Based Software

There are several software packages based on plane wave available for performing DFT, such as Vienna Ab initio Simulation Package (VASP), Qbox, Quantum ESPRESSO (QE), PEtot [28] and PWDFT [24]. VASP, Qbox, and QE are traditional software mainly used for simulations of medium-size atoms (hundreds of atoms) and PWDFT and PEtot are designed for large-size atoms (thousands of atoms). We list the related works in Table IV. It is important to note that the list provided in Table IV involve results obtained from different physical systems and computational resources, thus serving primarily as a general reference rather than a rigorous benchmarking. The current work emphasizes performance optimization tailored specifically for the Sunway architecture, highlighting scalability and capability for handling large-scale systems. A detailed

TABLE IV
LIST OF DIFFERENT PW-BASED DFT SOFTWARE

Software	Year	System	Atom	Machine	Scale (cores)
Qbox [10]	2017	Gold	9K	Trinity	300K
Qbox [18]	2006	Mo	1K	BlueGene/L	128K
VASP [26]	2018	Amorphous silicon	1K	Ohio Supercomputer and Tesla K40 GPU	not mentioned
PEtot [29]	2013	CdSe	1K	Cray XK6, IPE	256 GPUs
QE [15]	2017	Clycine, H ₂ O	768	Cray XC40	65K
PWDFT [24]	2017	Si	4K	Edision, Cori	8K
PWDFT	this work	Graphene, Si	16K	new Sunway	32K

comparison under consistent accuracy settings and computational conditions has been reported separately in PyP-WDFT [14].

To optimize plane-wave-based DFT calculation, one way is to port to the latest heterogeneous or homogeneous computing platforms [28], [30], [59], another way is to accelerate at the numerical algorithm level [24], [30], [31].

Recently, thanks to heterogeneous computing systems such as graphics process units (GPU) and many-core microarchitecture, those traditional software packages come to rejuvenate after being ported onto the new generation of supercomputers. In 2006, the winner of the Gordon Bell Prize extended the limit of Qbox on the BlueGene/L supercomputer to 1 K atoms, scaling up to 128 k cores on 65,536 nodes [18]. In 2017, researchers successfully ran Qbox [10] on 9418 nodes on Trinity with 8,800 atoms. In 2013, Jia [28] implemented PEtot on a multi-node GPU machine with x13-x22 speedups over the CPU calculations for a typical 512-atom system. Researchers ported VASP to Sunway TaihuLight [59] and achieved 2.90x to 4.48x speedup, but they only focused on small cases containing 32 Ge atoms and one Ru atom. It is significantly smaller than other works.

B. Other Research Works on the New Sunway Supercomputer

There are a lot of successful works that effectively utilize the massive computing resource of the new Sunway supercomputer [23], [46], [49], [60], for instance: (1) By implementing a novel sparse direct solver PEXSI, DGDFT extended the limit of adaptive local basis DFT calculation to 2.5 million atoms using 35.9 million cores, achieving 5% theoretical peak performance [23]. (2) A brand new computational framework to solve quantum many-body problems by Artificial Intelligence (AI) was proposed and adapted to the Sunway, scaling up to 40 million heterogeneous cores with high efficiency [60]. (3) Based on the abstraction of application and hierarchical hardware, researchers presented a unified programming model, UniQ, for multiple simulation methods. UniQ can generate execution with high performance without any human involvement. It scales to 399,360 nodes and achieves up to 28.59x speedup over state-of-the-art framework [58].

VII. CONCLUSION

In summary, we have significantly optimized the ability and performance of PWDFT on the new Sunway supercomputer. First, we reduce memory usage and improve memory bandwidth utilization by refactoring pseudopotential calculations and in-place data transforming, so that a single CG with 16 GB

memory is able to hold much more wavefunctions and a larger Hamiltonian matrix for larger systems. Second, we accelerate PWDFT by adopting CPEs, and granularity-aware parallelization to take full advantage of physical floating-point performance. Meanwhile, we also present multistage Allreduce to optimize network bandwidth utilization. As a result, PWDFT can now perform calculations of 4096 atoms using 64 CGs, 720 CGs for 11,520 atoms and 2,048 CGs for 16,384 atoms. These optimizations lead to a 64.8x speedup when simulating a silicon system comprising 4,096 atoms and utilizing 1,024 CGs. Additionally, our experiments demonstrate that our parallel design can achieve good strong scalability, even when utilizing a large number of cores.

ACKNOWLEDGMENT

The author thank the reviewers of ICPP 2023, IPDPS 2024, Euro-Par 2024, and TPDS for their feedback to strengthen this work.

REFERENCES

- [1] E. Angerson et al., "LAPACK: A portable linear algebra library for high-performance computers," in *Proc. 1990 ACM/IEEE Conf. Supercomputing*, 1990, pp. 2–11.
- [2] A. S. Banerjee, L. Lin, P. Suryanarayana, C. Yang, and J. E. Pask, "Two-level Chebyshev filter based complementary subspace method: Pushing the envelope of large-scale electronic structure calculations," *J. Chem. Theory Computation*, vol. 14, no. 6, pp. 2930–2946, 2018.
- [3] T. A. Barnes et al., "Improved treatment of exact exchange in quantum espresso," *Comput. Phys. Commun.*, vol. 214, pp. 52–58, 2017.
- [4] E. Cancès, M. Hassan, and L. Vidal, "Modified-operator method for the calculation of band diagrams of crystalline materials," *Math. Computation*, vol. 93, no. 347, pp. 1203–1245, 2024.
- [5] H. Cao and J. Chen, "Design and implementation of ShenWei universal C/C++," 2022, *arXiv:2208.00607*.
- [6] E. A. Carter, "Challenges in modeling materials properties without experimental input," *Science*, vol. 321, no. 5890, pp. 800–803, 2008.
- [7] Y. Chen, L. Zhang, H. Wang, and E. Weinan, "DeePKS: A comprehensive data-driven approach toward chemically accurate density functional theory," *J. Chem. Theory Computation*, vol. 17, no. 1, pp. 170–181, 2020.
- [8] Y. Chen et al., "DeePKS-kit: A package for developing machine learning-based chemically accurate energy and density functional models," *Comput. Phys. Commun.*, vol. 282, 2023, Art. no. 108520.
- [9] S. Das, P. Motamarri, V. Gavini, B. Turcksin, Y. W. Li, and B. Leback, "Fast, scalable and accurate finite-element based ab initio calculations using mixed precision computing: 46 PFLOPS simulation of a metallic dislocation system," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2019, pp. 1–11.
- [10] D. Ding, "A presentation on trinity performance," Aug. 2017. [Online]. Available: <https://www.osti.gov/biblio/1469055>
- [11] J. J. Dongarra, J. Du Croz, S. Hammarling, and I. S. Duff, "A set of level 3 basic linear algebra subprograms," *ACM Trans. Math. Softw.*, vol. 16, no. 1, pp. 1–17, 1990.
- [12] M. Frigo and S. G. Johnson, "FFTW: An adaptive software architecture for the FFT," in *Proc. 1998 IEEE Int. Conf. Acoust. Speech Signal Process.*, 1998, pp. 1381–1384.

- [13] M. J. Frisch, J. A. Pople, and J. S. Binkley, "Self-consistent molecular orbital methods 25. Supplementary functions for Gaussian basis sets," *J. Chem. Phys.*, vol. 80, no. 7, pp. 3265–3269, 1984.
- [14] J. Gao et al., "PyPWDFT: A lightweight python software for single-node 10k atom plane-wave density functional theory calculations," *J. Chem. Theory Computation*, vol. 21, pp. 2353–2370, 2025.
- [15] P. Giannozzi et al., "Advanced capabilities for materials modelling with quantum espresso," *J. Phys.: Condens. Matter*, vol. 29, no. 46, 2017, Art. no. 465901.
- [16] P. Giannozzi et al., "Quantum espresso: A modular and open-source software project for quantum simulations of materials," *J. Phys.: Condens. Matter*, vol. 21, no. 39, 2009, Art. no. 395502.
- [17] M. C. Groenenboom, R. M. Anderson, J. A. Wollmershauser, D. J. Horton, S. A. Policastro, and J. A. Keith, "Combined neural network potential and density functional theory study of TiAl₂O₅ surface morphology and oxygen reduction reaction overpotentials," *J. Phys. Chem. C*, vol. 124, no. 28, pp. 15171–15179, 2020.
- [18] F. Gygi et al., "Large-scale electronic structure calculations of high-Z metals on the BlueGene/L platform," in *Proc. 2006 ACM/IEEE Conf. Supercomputing*, 2006, pp. 45–es.
- [19] J. Hafner, "Ab-initio simulations of materials using VASP: Density-functional theory and beyond," *J. Comput. Chem.*, vol. 29, no. 13, pp. 2044–2078, 2008.
- [20] Y. Hasegawa et al., "First-principles calculations of electron states of a silicon nanowire with 100,000 atoms on the K computer," in *Proc. 2011 Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2011, pp. 1–11.
- [21] W. J. Hehre, R. F. Stewart, and J. A. Pople, "Self-consistent molecular-orbital methods. I. Use of Gaussian expansions of Slater-type atomic orbitals," *J. Chem. Phys.*, vol. 51, no. 6, pp. 2657–2664, 1969.
- [22] P. Hohenberg and W. Kohn, "Inhomogeneous electron gas," *Phys. Rev.*, vol. 136, no. 3B, 1964, Art. no. B864.
- [23] W. Hu et al., "2.5 million-atom ab initio electronic-structure simulation of complex metallic heterostructures with DGDFT," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2022, pp. 48–60.
- [24] W. Hu, L. Lin, A. S. Banerjee, E. Vecharynski, and C. Yang, "Adaptively compressed exchange operator for large-scale hybrid density functional calculations with applications to the adsorption of water on silicene," *J. Chem. Theory Computation*, vol. 13, no. 3, pp. 1188–1198, 2017.
- [25] J. Hutter, M. Iannuzzi, F. Schiffmann, and J. VandeVondele, "Wiley interdiscip," *Rev. Comput. Mol. Sci.*, vol. 4, no. 1, pp. 15–25, 2014.
- [26] D. Igram, B. Bhattacharai, P. Biswas, and D. A. Drabold, "Large and realistic models of amorphous silicon," *J. Non-Crystalline Solids*, vol. 492, pp. 27–32, 2018.
- [27] F. Jensen, "Polarization consistent basis sets. II. Estimating the Kohn–Sham basis set limit," *J. Chem. Phys.*, vol. 116, no. 17, pp. 7372–7379, 2002.
- [28] W. Jia et al., "The analysis of a plane wave pseudopotential density functional theory code on a GPU machine," *Comput. Phys. Commun.*, vol. 184, no. 1, pp. 9–18, 2013.
- [29] W. Jia et al., "Fast plane wave density functional theory molecular dynamics calculations on multi-GPU machines," *J. Comput. Phys.*, vol. 251, pp. 102–115, 2013.
- [30] W. Jia, L.-W. Wang, and L. Lin, "Parallel transport time-dependent density functional theory calculations with hybrid functional on summit," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2019, pp. 1–23.
- [31] Q. Jiang et al., "Accelerating parallel first-principles excited-state calculation by low-rank approximation with K-means clustering," in *Proc. 51st Int. Conf. Parallel Process.*, 2022, pp. 1–11.
- [32] T. Kocabaş, M. Keçeli, Á. Vázquez-Mayagoitia, and C. Sevik, "Gaussian approximation potentials for accurate thermal properties of two-dimensional materials," *Nanoscale*, vol. 15, no. 19, pp. 8772–8780, 2023.
- [33] G. Kresse and J. Furthmüller, "Efficient iterative schemes for ab initio total-energy calculations using a plane-wave basis set," *Phys. Rev. B*, vol. 54, no. 16, 1996, Art. no. 11169.
- [34] T. D. Kühne et al., "CP2K: An electronic structure and molecular dynamics software package-quickstep: Efficient and accurate electronic structure calculations," *J. Chem. Phys.*, vol. 152, no. 19, 2020, Art. no. 194103.
- [35] H. Li et al., "Deep-learning density functional theory hamiltonian for efficient ab initio electronic-structure calculation," *Nature Comput. Sci.*, vol. 2, no. 6, pp. 367–377, 2022.
- [36] D. Lu et al., "86 PFLOPS deep potential molecular dynamics simulation of 100 million atoms with ab initio accuracy," *Comput. Phys. Commun.*, vol. 259, 2021, Art. no. 107624.
- [37] V. Milman et al., "Electronic structure, properties, and phase stability of inorganic crystals: A pseudopotential plane-wave study," *Int. J. Quantum Chem.*, vol. 77, no. 5, pp. 895–910, 2000.
- [38] P. Motamarri, S. Das, S. Rudraraju, K. Ghosh, D. Davydov, and V. Gavini, "DFT-FE—A massively parallel adaptive finite-element code for large-scale density functional theory calculations," *Comput. Phys. Commun.*, vol. 246, 2020, Art. no. 106853.
- [39] A. Nakata et al., "Large scale and linear scaling DFT with the CONQUEST code," *J. Chem. Phys.*, vol. 152, no. 16, 2020, Art. no. 164112.
- [40] NVIDIA, "NVIDIA Tesla V100 GPU architecture. White Paper," 2017. [Online]. Available: <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>
- [41] NVIDIA, "NVIDIA A100 tensor core GPU architecture. White Paper," 2020. [Online]. Available: <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>
- [42] M. C. Payne, T. A. Arias, and J. D. Joannopoulos, "Iterative minimization techniques for ab initio total-energy calculations: Molecular dynamics and conjugate gradients," *Rev. Modern Phys.*, vol. 64, no. 4, pp. 1045–1097, 1992.
- [43] A. Pedersen, L. Pizzagalli, and H. Jónsson, "Optimal atomic structure of amorphous silicon obtained from density functional theory calculations," *New J. Phys.*, vol. 19, no. 6, 2017, Art. no. 063018.
- [44] M. Sato, Y. Kodama, M. Tsuji, and T. Odajima, "Co-design and system for the supercomputer "Fugaku"," *IEEE Micro*, vol. 42, no. 2, pp. 26–34, Mar./Apr. 2022.
- [45] H. Shang et al., "Extreme-scale ab initio quantum raman spectra simulations on the leadership HPC system in China," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2021, pp. 1–13.
- [46] H. Shang et al., "Large-scale simulation of quantum computational chemistry on a new Sunway supercomputer," 2022, *arXiv:2207.03711*.
- [47] H. Wang et al., "DeePMD-kit: A deep learning package for many-body potential energy representation and molecular dynamics," *Comput. Phys. Commun.*, vol. 228, pp. 178–184, 2018.
- [48] L.-W. Wang et al., "Linearly scaling 3D fragment method for large-scale electronic structure calculations," in *Proc. 2008 ACM/IEEE Conf. Supercomputing*, 2008, pp. 1–10.
- [49] Y. Wang, H. Cao, Z. Ma, W. Yin, and W. Chen, "Scaling graph 500 SSSP to 140 trillion edges with over 40 million cores," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2022, pp. 1–15.
- [50] Y. Wang et al., "DeepH-2: Enhancing deep-learning electronic structure via an equivariant local-coordinate transformer," 2024, *arXiv:2401.17015*.
- [51] J. Wells et al., "Announcing supercomputer summit," Oak Ridge National Lab.(ORNL), Oak Ridge, TN, USA, 2016.
- [52] Q. Xu, M. Del Ben, M. S. Okay, M. Choi, K. Z. Ibrahim, and B. M. Wong, "Velocity-gauge real-time time-dependent density functional tight-binding for large-scale condensed matter systems," *J. Chem. Theory Computation*, vol. 19, no. 22, pp. 7989–7997, 2023.
- [53] Q. Xu et al., "SPARC: Simulation package for ab-initio real-space calculations," *SoftwareX*, vol. 15, 2021, Art. no. 100709.
- [54] M. Yin and M. L. Cohen, "Theory of ab initio pseudopotential calculations," *Phys. Rev. B*, vol. 25, no. 12, 1982, Art. no. 7403.
- [55] J. Zeng et al., "DeePMD-kit v2: A software package for deep potential models," *J. Chem. Phys.*, vol. 159, no. 5, 2023, Art. no. 054801.
- [56] C. Zeni et al., "A generative model for inorganic materials design," *Nature*, vol. 639, pp. 624–632, 2025.
- [57] C. Zhang et al., "Advancing nonadiabatic molecular dynamics simulations in solids with E(3) equivariant deep neural hamiltonians," *Nature Commun.*, vol. 16, no. 1, 2025, Art. no. 2033.
- [58] C. Zhang, H. Wang, Z. Ma, L. Xie, Z. Song, and J. Zhai, "UniQ: A unified programming model for efficient quantum circuit simulation," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2022, pp. 692–707.
- [59] H. Zhao and C. Zhang, "Porting and optimizing VASP on the SW26010," in *Proc. Int. Workshops Algorithms Architectures Parallel Process.*, Springer, 2018, Art. no. 17.
- [60] X. Zhao et al., "AI for quantum mechanics: High performance quantum many-body simulations via deep learning," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2022, pp. 677–691.
- [61] Z. Zhao, M. Marsman, F. Wende, and J. Kim, "Performance of hybrid MPI/OpenMP VASP on cray XC40 based on Intel knights landing many integrated core architecture," in *CUG Proc.*, 2017.
- [62] Q. Zhu, H. Luo, C. Yang, M. Ding, W. Yin, and X. Yuan, "Enabling and scaling the HPCG benchmark on the newest generation Sunway supercomputer with 42 million heterogeneous cores," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2021, Art. no. 57.



Qingcai Jiang received the BS degree from the University of Science and Technology of China, in 2019. He is currently working toward the PhD degree with the University of Science and Technology of China, where he is advised by Prof. Hong An. From October 2023 to October 2024, he spent one year as an academic guest with ETH Zurich, hosted by Prof. Onur Mutlu. His research interests include parallel computing, workload characterization, and system support for processing-in-memory architectures. For more information, please see his webpage at <https://qcjiang.github.io/>.



Wei Hu received the PhD degree in theoretical and computational chemistry from the University of Science and Technology of China (USTC), in 2013. Then, he joined the Computational Research Division of Lawrence Berkeley National Laboratory as a postdoctoral fellow for developing high performance computing (HPC) density functional theory (DFT) software. He is currently a professor with the Hefei National Laboratory for Physical Sciences, Microscale (HFNL) of USTC. His research interest focuses on the method development and scientific applications of large-scale DFT calculations.



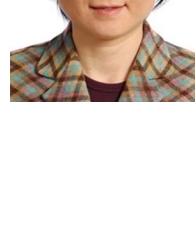
Zhenwei Cao received the BS degree from Wuhan University, in 2021, and the MS degree from the University of Science and Technology of China, in 2024. Since July 2024, he has been working with Kuaishou Technology, specializing in system software and performance optimization.



Hong An (Member, IEEE) received the PhD degree in computer science from the University of Science and Technology of China (USTC), Hefei, in 2000. She is a full professor with USTC, Hefei, and the director of the Advanced Computer System Architecture Laboratory, USTC. Her main research areas are parallel computer architecture for energy-efficient high-performance computing, parallel programming, and operating system design. She is a member of the CCF and ACM.



Junshi Chen received the PhD degree in computer science from the University of Science and Technology of China (USTC), Hefei, in 2020. He is an assistant professor with USTC, Hefei. His research interests include high-performance computing and computer architecture. He is a member of the CCF.



Jinlong Yang received the PhD degree in condensed matter physics from the University of Science and Technology of China (USTC), in 1991. He is currently a professor with USTC. He became the dean with the School of Chemistry and Materials Science, USTC, in 2009 and the vice president with USTC, in 2018. His research interest focuses on the development of first-principles methods and their applications to clusters, nanostructures, solid materials, surfaces, and interfaces.



Xinming Qin received the BS and PhD degrees from the University of Science and Technology of China (USTC), in 2009 and 2018. He is currently an associate researcher with Hefei National Laboratory for Physical Sciences at the Microscale (HFNL). His research interests include reduced-scaling numerical algorithms, high-performance computational chemistry software, and large-scale materials simulations using high performance computing.

